

TP Sécuriser une mini-application utilisateur

Objectifs :

- Comprendre les **rôles** et le **contrôle d'accès (RBAC)**.
- Simuler une **authentification simple avec mot de passe + second facteur**.
- Découvrir le **hachage de mot de passe**.
- Découvrir le **chiffrement AES des données sensibles**.
- Identifier les **failles dans les fichiers de configuration**.

Soit le fichier JSON suivant :

```
{
  "users": [
    { "username": "admin1", "role": "admin" },
    { "username": "user1", "role": "user" },
    { "username": "guest1", "role": "guest" }
  ],
  "permissions": {
    "read": ["admin", "user", "guest"],
    "write": ["admin", "user"],
    "delete": ["admin"]
  }
}
```

Questions :

- Qui peut **lire** ?
- Qui peut **supprimer** ?
- Que se passe-t-il si user1 veut supprimer un compte ?

Partie 2 :

1. Soit le scénario suivant:

// Mot de passe en clair

```
const motDePasseUtilisateur = "azerty123";  
const motDePasseTape = "azerty123"; // simulé par l'utilisateur
```

// Facteur 2 simulé

```
const codeEnvoye = "123456";  
const codeTape = "123456";
```

Questions :

- Vérifier que le mot de passe est bon.
- Vérifier que le code 2FA est correct.
- Écrire en pseudo-code :

SI motDePasseTape == motDePasseUtilisateur ET codeTape == codeEnvoye ALORS connecter
SINON refuser

3/

- Montrer comment **hachage du mot de passe avec bcrypt** fonctionnerait dans un vrai cas.

```
const bcrypt = require('bcrypt');  
const hashed = await bcrypt.hash("azerty123", 10);  
const isMatch = await bcrypt.compare("azerty123", hashed);
```

Partie 3 : Cryptage AES simple (carte bancaire)

1. Donner un numéro fictif : 4111 1111 1111 1111
2. Chiffrer avec un outil en ligne ou une démo Node.js :

```
const crypto = require("crypto");  
const key = crypto.randomBytes(32);  
const iv = crypto.randomBytes(16);  
  
const cipher = crypto.createCipheriv("aes-256-cbc", key, iv);  
let encrypted = cipher.update("4111111111111111", "utf8", "hex");  
encrypted += cipher.final("hex");  
  
console.log("Encrypted:", encrypted);
```

Questions :

- Pourquoi on ne peut rien faire avec les données chiffrées ?
- Pourquoi il faut **protéger la clé** ?

Partie 4 : Fichier de configuration non sécurisé

1. Montrer ce fichier .env mal conçu :

DB_PASSWORD=root123

API_KEY=cle-secrete

2. ➤ **Question** :

- Pourquoi c'est dangereux de publier ce fichier ?
- Que faudrait-il faire pour sécuriser ce fichier (ex: .gitignore, lecture seule, etc.) ?